

# **Competitive Programming**

**Section 42 Editorial: Advanced Data Structures Mixed Contest**

Competitive Programming Course

2026-08-14

# Table of contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Category coverage</b>                              | <b>4</b>  |
| <b>2</b> | <b>Editorial overview</b>                             | <b>5</b>  |
| 2.1      | Problem-specific proof map . . . . .                  | 5         |
| 2.2      | Detailed solution guide . . . . .                     | 5         |
| <b>3</b> | <b>A. Dynamic Order Statistics</b>                    | <b>7</b>  |
| 3.1      | Restatement . . . . .                                 | 7         |
| 3.2      | Observations, trace, and algorithm . . . . .          | 7         |
| 3.3      | Correctness proof and complexity . . . . .            | 7         |
| 3.4      | Full C++ solution . . . . .                           | 7         |
| 3.5      | Full Python solution . . . . .                        | 7         |
| 3.6      | Tests that matter . . . . .                           | 8         |
| <b>4</b> | <b>B. Range Add Range Sum</b>                         | <b>9</b>  |
| 4.1      | Restatement . . . . .                                 | 9         |
| 4.2      | Observations, trace, and algorithm . . . . .          | 9         |
| 4.3      | Correctness proof and complexity . . . . .            | 9         |
| 4.4      | Full C++ solution . . . . .                           | 9         |
| 4.5      | Full Python solution . . . . .                        | 9         |
| 4.6      | Tests that matter . . . . .                           | 10        |
| <b>5</b> | <b>C. Static Rectangle Count</b>                      | <b>11</b> |
| 5.1      | Restatement . . . . .                                 | 11        |
| 5.2      | Observations, trace, and algorithm . . . . .          | 11        |
| 5.3      | Correctness proof and complexity . . . . .            | 11        |
| 5.4      | Full C++ solution . . . . .                           | 11        |
| 5.5      | Full Python solution . . . . .                        | 11        |
| 5.6      | Tests that matter . . . . .                           | 12        |
| <b>6</b> | <b>D. Nested Ranges Count</b>                         | <b>13</b> |
| 6.1      | Restatement and constraints . . . . .                 | 13        |
| 6.2      | Ordering and worked trace . . . . .                   | 13        |
| 6.3      | Algorithm . . . . .                                   | 14        |
| 6.4      | Correctness proof . . . . .                           | 14        |
| 6.5      | Complexity, implementation notes, and tests . . . . . | 14        |
| 6.6      | Full C++ solution . . . . .                           | 14        |
| 6.7      | Full Python solution . . . . .                        | 16        |
| <b>7</b> | <b>Progressive challenge solutions</b>                | <b>19</b> |
| <b>8</b> | <b>E. Subtree Add, Point Query</b>                    | <b>20</b> |
| <b>9</b> | <b>F. Dynamic Path Maximum</b>                        | <b>23</b> |

|                               |    |
|-------------------------------|----|
| 10 G. Subtree Value Count     | 26 |
| 11 H. Connectivity Countdown  | 29 |
| 12 I. Persistent Version Sums | 31 |
| 13 J. Subtree K-th Smallest   | 34 |
| 14 K. Sparse Rectangle Sums   | 37 |
| 15 L. Path Add, Path Maximum  | 40 |
| 16 Testing discussion         | 44 |
| 17 What to carry forward      | 45 |

# 1 Category coverage

| Category                                     | Problem                     |
|----------------------------------------------|-----------------------------|
| order statistics with Fenwick binary lifting | A. Dynamic Order Statistics |
| range-add/range-sum with two Fenwick trees   | B. Range Add Range Sum      |
| offline rectangle counting                   | C. Static Rectangle Count   |

All required categories are present locally. D. Nested Ranges Count is an additional sorting-and-Fenwick synthesis problem.

## 2 Editorial overview

### 2.1 Problem-specific proof map

**A. Dynamic Order Statistics.** A Fenwick prefix is exactly the number of stored values at most that index. Fenwick binary lifting finds the first prefix whose count reaches  $k$ , hence the  $k$ -th value, in  $O(\log m)$  per operation.

**B. Range Add Range Sum.** The two-tree formula is the prefix sum of the difference array after distributing each range update over its affected positions. Subtracting two such prefixes therefore gives exactly the requested range sum, with  $O(\log n)$  updates and queries.

**C. Static Rectangle Count.** Inclusion-exclusion turns a rectangle into four dominance counts. During the  $x$ -sweep, the Fenwick tree contains precisely points with  $x \leq X$ , so its  $y$ -prefix produces each dominance count in  $O(\log n)$ .

**D. Nested Ranges Count.** Sort intervals by left endpoint ascending and right endpoint descending. A Fenwick tree over right endpoints counts the intervals already seen in the relevant sweep direction; process equal interval pairs together so identical intervals count one another.

### 2.2 Detailed solution guide

**A. Dynamic Order Statistics.** A Fenwick tree over value coordinates stores frequencies.  $\text{sum}(x)$  is the number of elements  $\leq x$ . Insert and erase update one coordinate by  $+1$  or  $-1$ . The  $k$ -th element is the smallest coordinate whose prefix sum is at least  $k$ ; binary lifting inside the Fenwick tree finds that coordinate in logarithmic time.

**B. Range Add, Range Sum.** Difference arrays turn a range add into two point events, but range-sum queries need one more accumulated moment. Maintain two Fenwick trees  $B1$  and  $B2$  so that after adding  $\text{delta}$  to  $[l, r]$ ,  $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$ . Then  $\text{sum}(l, r)$  is  $\text{prefix}(r) - \text{prefix}(l-1)$ .

**C. Offline Rectangle Counting.** Count points in rectangle  $[x1, x2] \times [y1, y2]$  by inclusion-exclusion of four dominance queries  $\text{count}(x \leq X, y \leq Y)$ . Sort points and queries by  $X$ , sweep from left to right, and insert active point  $y$ -coordinates into a Fenwick tree. The  $y$ -prefix sum then answers the dominance query.

**Common pitfalls.** Coordinate-compress values before using a Fenwick tree when coordinates are large. For range sums, update both trees with matching signs. For rectangles, inclusion-exclusion signs are  $+$   $-$   $-$   $+$ .

All three problems reduce a seemingly richer query to prefix queries.

- A stores frequencies in a Fenwick tree. Counts are prefix sums;  $k$ -th order statistic is the first index whose prefix sum reaches  $k$ .
- B stores a range-added array through two Fenwick trees. The formula  $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$  converts range additions into prefix sums of the final array.

- C expands each rectangle into four dominance queries  $x \leq X$ ,  $y \leq Y$ , then sweeps points by x-coordinate and answers y-prefix counts with a Fenwick tree.

The recurring invariant is that the Fenwick tree contains exactly the active objects for the current prefix threshold.

## 3 A. Dynamic Order Statistics

### 3.1 Restatement

Maintain a multiset over values  $1..m$ : insert, erase one copy, k-th, and count  $\leq x$ .

### 3.2 Observations, trace, and algorithm

Keep multiplicities in a Fenwick tree. After inserts 3,5,3, rank two is 3 and count  $\leq 4$  is two; after erasing one 3, rank two is 5. Insert/erase update one index, count is a prefix sum, and kth is the first prefix reaching k by Fenwick binary lifting; reject an invalid k.

### 3.3 Correctness proof and complexity

Fenwick prefixes equal cumulative multiplicities, so the first prefix reaching k is exactly the kth multiset value. Every operation is  $O(\log m)$  and memory is  $O(m)$ . QED.

### 3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int> b; BIT(int n):n(n),b(n+1){} void add(int
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int m,q;if(!(cin>>m>>q)) return 0; E
```

### 3.5 Full Python solution

```
import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
    def kth(self,k):
        if k<1 or k>self.sum(self.n): return -1
        pos=0; step=1<<self.n.bit_length()
```

```

        while step:
            nxt=pos+step
            if nxt<=self.n and self.bit[nxt]<k: pos=nxt; k-=self.bit[nxt]
            step//=2
        return pos+1
def main():
    data=list(map(int,sys.stdin.buffer.read().split()));
    if not data: return
    m,q=data[0],data[1]; bit=BIT(m); cnt=[0]*(m+1); idx=2; out=[]
    for _ in range(q):
        t,x=data[idx],data[idx+1]; idx+=2
        if t==1: cnt[x]+=1; bit.add(x,1)
        elif t==2:
            if cnt[x]: cnt[x]-=1; bit.add(x,-1)
        elif t==3: out.append(str(bit.kth(x)))
        else: out.append(str(bit.sum(x)))
    print("\n".join(out))
if __name__=="__main__": main()

```

## 3.6 Tests that matter

Test duplicate insert/erase and invalid ranks, negative/full-range updates, and points on rectangle boundaries with duplicate coordinates.

## 4 B. Range Add Range Sum

### 4.1 Restatement

Support range addition and range-sum queries.

### 4.2 Observations, trace, and algorithm

An add  $x$  on  $[l, r]$  writes  $+x$  at  $l$  and  $-x$  at  $r+1$  in a difference tree. A second weighted tree gives  $\text{prefix}(i) = i \cdot \text{sum}(B1, i) - \text{sum}(B2, i)$ . Adding ten on  $[2, 4]$  makes the added contribution through  $i=3$  equal  $3 \cdot 10 - 10 = 20$ .

### 4.3 Correctness proof and complexity

Summing  $B1$  reconstructs each pending add; summing those reconstructed values gives the weighted formula. Prefix subtraction isolates a range. Updates and queries are  $O(\log n)$ , memory  $O(n)$ . QED.

### 4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<long long>b; BIT(int n):n(n),b(n+2){} void add
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; E
```

### 4.5 Full Python solution

```
import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+2)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
```

```

if not data: return
n,q=data[0],data[1]; a=data[2:2+n]; b1=BIT(n+1); b2=BIT(n+1)
def add_range(l,r,x): b1.add(l,x); b1.add(r+1,-x); b2.add(l,x*(l-1)); b2.add(r+1,-x*r)
def pref(i): return b1.sum(i)*i-b2.sum(i)
for i,x in enumerate(a,1): add_range(i,i,x)
idx=2+n; out=[]
for _ in range(q):
    t=data[idx]; idx+=1
    if t==1:
        l,r,x=data[idx],data[idx+1],data[idx+2]; idx+=3; add_range(l,r,x)
    else:
        l,r=data[idx],data[idx+1]; idx+=2; out.append(str(pref(r)-pref(l-1)))
print("\n".join(out))
if __name__=="__main__": main()

```

## 4.6 Tests that matter

Test duplicate insert/erase, invalid ranks, negative/full-range updates, and points on inclusive rectangle boundaries with duplicate coordinates.

## 5 C. Static Rectangle Count

### 5.1 Restatement

Given points and rectangle queries, count points inside each rectangle.

### 5.2 Observations, trace, and algorithm

Turn a rectangle into x-prefix events at  $x_2$  (+) and  $x_1-1$  (-). Sort events and points by x; a compressed-y Fenwick tree contains points already passed. For points (1,1),(2,3),(4,2), rectangle [1,3]x[1,3] sees two points at  $x=3$  and zero at  $x=0$ .

### 5.3 Correctness proof and complexity

At an event, the tree contains exactly points with x at most its threshold; its y-prefix difference counts the requested y interval. Subtracting events is rectangle inclusion-exclusion. Cost is  $O((n+q)\log(n+q))$ , memory  $O(n+q)$ . QED.

### 5.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int>b; BIT(int n):n(n),b(n+1){} void add(int i, int x){
struct E{int x,y1,y2,id,sgn; bool operator<(const E&o)const{return x<o.x;}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; v
```

### 5.5 Full Python solution

```
import sys,bisect
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
```

```

data=list(map(int,sys.stdin.buffer.read().split()))
if not data: return
n,q=data[0],data[1]; idx=2; pts=[]; ys=[]
for _ in range(n): x,y=data[idx],data[idx+1]; idx+=2; pts.append((x,y)); ys.append(y)
events=[]; ans=[0]*q
for qi in range(q):
    x1,y1,x2,y2=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; ys+= [y1,y2]
    events.append((x2,y1,y2,qi,1)); events.append((x1-1,y1,y2,qi,-1))
ys=sorted(set(ys)); pts.sort(); events.sort(); bit=BIT(len(ys)); p=0
for x,y1,y2,qi,sgn in events:
    while p<n and pts[p][0]<=x:
        bit.add(bisect.bisect_left(ys,pts[p][1])+1,1); p+=1
    l=bisect.bisect_left(ys,y1)+1; r=bisect.bisect_right(ys,y2)
    ans[qi]+=sgn*(bit.sum(r)-bit.sum(l-1))
print("\n".join(map(str,ans)))
if __name__=="__main__": main()

```

## 5.6 Tests that matter

Test duplicate insert/erase, invalid ranks, negative/full-range updates, and points on inclusive rectangle boundaries with duplicate coordinates.

## 6 D. Nested Ranges Count

### 6.1 Restatement and constraints

For every one of  $n$  closed intervals  $[l_i, r_i]$ , count two quantities:

1. how many *other* intervals it contains; and
2. how many *other* intervals contain it.

Interval  $i$  contains interval  $j$  exactly when  $l_i \leq l_j$  and  $r_j \leq r_i$ . Identical intervals therefore contain each other. With up to 200,000 intervals, comparing all ordered pairs is quadratic.

### 6.2 Ordering and worked trace

Sort interval records by (left ascending, right descending). The descending tie-break is essential: among intervals that begin together, a larger right endpoint can contain a smaller one.

For intervals

```
0: [1,6]
1: [2,5]
2: [2,5]
3: [3,4]
```

the contains counts are  $[3,2,2,0]$ : interval 0 contains all other intervals; each copy of  $[2,5]$  contains the other copy and  $[3,4]$ . The contained-by counts are  $[0,2,2,3]$ .

Compress right endpoints. To count **contained-by**, sweep the sorted list left to right. Earlier records have a left endpoint no larger than the current one. Among them, the number with right endpoint at least  $r$  is `processed - BIT.prefix(rank(r)-1)`.

To count **contains**, sweep the same ordering right to left. Already processed records have a left endpoint no smaller than the current one. The number with right endpoint at most  $r$  is `BIT.prefix(rank(r))`.

Equal pairs such as the two  $[2,5]$  records must be processed as a group in both sweeps. Compute the common answer against previously processed intervals, then add `group_size-1` because every duplicate contains, and is contained by, every other duplicate. Only then insert the whole group into the Fenwick tree.

## 6.3 Algorithm

```
sort records by (left increasing, right decreasing)
compress all right endpoints

for each equal-pair group from left to right:
    base = processed - BIT.prefix(rank(right) - 1)
    contained_by[index] = base + group_size - 1 for every index in group
    insert group_size copies of rank(right) into BIT

clear BIT
for each equal-pair group from right to left:
    base = BIT.prefix(rank(right))
    contains[index] = base + group_size - 1 for every index in group
    insert group_size copies of rank(right) into BIT
```

## 6.4 Correctness proof

Consider the forward sweep. Before a group is processed, the ordering ensures every inserted interval has left endpoint at most the group's left endpoint. The Fenwick tree counts their right endpoints. Subtracting the count strictly below the group's right endpoint from the number processed leaves exactly those with right endpoint at least it, which are exactly the previously inserted intervals containing this group. Equal-pair additions count every other identical interval, and delaying their insertion prevents double counting. Thus `contained_by` is correct.

For the reverse sweep, every inserted interval has left endpoint at least the current group's left endpoint. The Fenwick prefix through its right endpoint counts exactly those whose right endpoint is at most it, which are exactly the inserted intervals contained by this group. The same equal-pair argument adds the identical intervals exactly once. Thus `contains` is correct. QED.

## 6.5 Complexity, implementation notes, and tests

Sorting/compression cost  $O(n \log n)$ , and each Fenwick operation costs  $O(\log n)$ , so total time is  $O(n \log n)$  with  $O(n)$  memory. Keep original indices in the sorted records so answers can be printed in input order.

Test identical intervals, intervals sharing only one endpoint, disjoint intervals, and a fully nested chain. The common error is to update a duplicate immediately; that makes the first duplicate see none of its peers while later duplicates see too many.

## 6.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;
    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int index, int delta) {
        for (int n = (int)bit.size(); index < n; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }
};

struct Interval {
    long long left, right;
    int index;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<Interval> ranges(n);
    vector<long long> rights;
    for (int i = 0; i < n; i++) {
        cin >> ranges[i].left >> ranges[i].right;
        ranges[i].index = i;
        rights.push_back(ranges[i].right);
    }
    sort(rights.begin(), rights.end());
    rights.erase(unique(rights.begin(), rights.end()), rights.end());
    sort(ranges.begin(), ranges.end(), [](const Interval& a, const Interval& b) {
        if (a.left != b.left) return a.left < b.left;
        return a.right > b.right;
    });
    auto rank_of = [&](long long right) {
        return int(lower_bound(rights.begin(), rights.end(), right) - rights.begin()) + 1;
    };

    vector<int> contains(n), contained_by(n);
    Fenwick bit((int)rights.size());
    int processed = 0;

```

```

for (int start = 0; start < n;) {
    int end = start + 1;
    while (end < n && ranges[end].left == ranges[start].left &&
           ranges[end].right == ranges[start].right) {
        end++;
    }
    int size = end - start;
    int rank = rank_of(ranges[start].right);
    int base = processed - bit.sum(rank - 1);
    for (int i = start; i < end; i++) contained_by[ranges[i].index] = base + size - 1;
    bit.add(rank, size);
    processed += size;
    start = end;
}

bit = Fenwick((int)rights.size());
for (int end = n; end > 0;) {
    int start = end - 1;
    while (start > 0 && ranges[start - 1].left == ranges[end - 1].left &&
           ranges[start - 1].right == ranges[end - 1].right) {
        start--;
    }
    int size = end - start;
    int rank = rank_of(ranges[start].right);
    int base = bit.sum(rank);
    for (int i = start; i < end; i++) contains[ranges[i].index] = base + size - 1;
    bit.add(rank, size);
    end = start;
}

for (int i = 0; i < n; i++) cout << contains[i] << (i + 1 == n ? '\n' : ' ');
for (int i = 0; i < n; i++) cout << contained_by[i] << (i + 1 == n ? '\n' : ' ');
return 0;
}

```

## 6.7 Full Python solution

```

import bisect
import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index < len(self.bit):
            self.bit[index] += delta

```

```

        index += index & -index

def prefix_sum(self, index: int) -> int:
    total = 0
    while index:
        total += self.bit[index]
        index -= index & -index
    return total

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    ranges = [(data[2 * i + 1], data[2 * i + 2], i) for i in range(n)]
    rights = sorted({right for _, right, _ in ranges})
    ranges.sort(key=lambda interval: (interval[0], -interval[1]))

    def rank_of(right: int) -> int:
        return bisect.bisect_left(rights, right) + 1

    contains = [0] * n
    contained_by = [0] * n

    bit = Fenwick(len(rights))
    processed = 0
    start = 0
    while start < n:
        end = start + 1
        while end < n and ranges[end][:2] == ranges[start][:2]:
            end += 1
        group_size = end - start
        rank = rank_of(ranges[start][1])
        base = processed - bit.prefix_sum(rank - 1)
        for _, _, index in ranges[start:end]:
            contained_by[index] = base + group_size - 1
        bit.add(rank, group_size)
        processed += group_size
        start = end

    bit = Fenwick(len(rights))
    end = n
    while end:
        start = end - 1
        while start > 0 and ranges[start - 1][:2] == ranges[end - 1][:2]:
            start -= 1
        group_size = end - start
        rank = rank_of(ranges[start][1])
        base = bit.prefix_sum(rank)
        for _, _, index in ranges[start:end]:

```

```
        contains[index] = base + group_size - 1
    bit.add(rank, group_size)
    end = start

    print(" ".join(map(str, contains)))
    print(" ".join(map(str, contained_by)))

if __name__ == "__main__":
    main()
```

## **7 Progressive challenge solutions**

## 8 E. Subtree Add, Point Query

An Euler tour assigns every subtree one contiguous interval  $[\text{tin}[u], \text{tout}[u]]$ . Represent a subtree addition as two changes in a Fenwick difference array:  $+x$  at  $\text{tin}[u]$  and  $-x$  after  $\text{tout}[u]$ . Its prefix sum at  $\text{tin}[v]$  is exactly the total of updates whose subtree intervals contain  $v$ ; add that to the original value. The interval property proves correctness. Preprocessing is  $O(n)$  and each operation is  $O(\log n)$ .

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> tin(n), tout(n), par(n, -1), it(n);
    int timer = 0;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        if (!it[u])
            tin[u] = ++timer;
        if (it[u] < (int)g[u].size()) {
            int v = g[u][it[u]++];
            if (v == par[u])
                continue;
            par[v] = u;
            st.push_back(v);
        } else {
            tout[u] = timer;
            st.pop_back();
        }
    }
}
```

```

vector<ll> bit(n + 2);
auto add = [&](int i, ll x) {
    for (; i <= n + 1; i += i & -i)
        bit[i] += x;
};
auto get = [&](int i) {
    ll s = 0;
    for (; i; i -= i & -i)
        s += bit[i];
    return s;
};
while (q--) {
    char c;
    int u;
    cin >> c >> u;
    --u;
    if (c == 'A') {
        ll x;
        cin >> x;
        add(tin[u], x);
        add(tout[u] + 1, -x);
    } else
        cout << a[u] + get(tin[u]) << '\n';
}
}

```

```

import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]));g=[[]for _ in range(n)];p=2+n
    for _ in range(n-1):
        u,v=int(d[p])-1,int(d[p+1])-1;p+=2;g[u].append(v);g[v].append(u)
    tin=[0]*n;tout=[0]*n;parent=[-1]*n;timer=0;stack=[(0,0)]
    while stack:
        u,state=stack.pop()
        if state==0:
            timer+=1;tin[u]=timer;stack.append((u,1))
            for v in reversed(g[u]):
                if v!=parent[u]:parent[v]=u;stack.append((v,0))
        else:tout[u]=timer
    bit=[0]*(n+2)
    def add(i,x):
        while i<=n+1:bit[i]+=x;i+=i&-i
    def get(i):
        s=0
        while i:s+=bit[i];i-=i&-i
        return s
    out=[]
    for _ in range(q):
        c=d[p];u=int(d[p+1])-1;p+=2

```

```
        if c==b'A':x=int(d[p]);p+=1;add(tin[u],x);add(tout[u]+1,-x)
        else:out.append(str(a[u]+get(tin[u])))
    print('\n'.join(out))
if __name__=="__main__":main()
```

## 9 F. Dynamic Path Maximum

Heavy-light decomposition partitions any tree path into  $O(\log n)$  contiguous segments of one base array. A segment tree stores the maximum in that array; assignments are point updates. While the chain heads differ, consume the deeper head-to-vertex segment, then finish the final common-chain segment. These pieces are disjoint and concatenate to the original path, so their maximum is the path maximum. Updates cost  $O(\log n)$ , queries  $O(\log^2 n)$ , and memory  $O(n)$ .

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<int> a(n), p(n, -1), dep(n), sz(n, 1), heavy(n, -1), head(n), pos(n),
        ord = {0};
    for (int &x : a)
        cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    for (int i = 0; i < (int)ord.size(); i++) {
        int u = ord[i];
        for (int v : g[u])
            if (v != p[u])
                p[v] = u, dep[v] = dep[u] + 1, ord.push_back(v);
    }
    for (int z = n - 1; z; z--) {
        int u = ord[z];
        sz[p[u]] += sz[u];
        if (heavy[p[u]] < 0 || sz[u] > sz[heavy[p[u]]])
            heavy[p[u]] = u;
    }
    int timer = 0;
    vector<pair<int, int>> todo = {{0, 0}};
    while (!todo.empty()) {
        auto [u, h] = todo.back();
        todo.pop_back();
```

```

    for (int x = u; x != -1; x = heavy[x]) {
        head[x] = h;
        pos[x] = timer++;
        for (int v : g[x])
            if (v != p[x] && v != heavy[x])
                todo.push_back({v, v});
    }
}

int z = 1;
while (z < n)
    z *= 2;
vector<int> t(2 * z, INT_MIN);
for (int u = 0; u < n; u++)
    t[z + pos[u]] = a[u];
for (int i = z - 1; i; i--)
    t[i] = max(t[2 * i], t[2 * i + 1]);
auto query = [&](int l, int r) {
    int ans = INT_MIN;
    for (l += z, r += z + 1; l < r; l /= 2, r /= 2) {
        if (l & 1)
            ans = max(ans, t[l++]);
        if (r & 1)
            ans = max(ans, t[--r]);
    }
    return ans;
};

while (q--) {
    char c;
    int u, v;
    cin >> c >> u >> v;
    --u;
    if (c == 'U') {
        int x = z + pos[u];
        t[x] = v;
        for (x /= 2; x; x /= 2)
            t[x] = max(t[2 * x], t[2 * x + 1]);
    } else {
        --v;
        int ans = INT_MIN;
        while (head[u] != head[v]) {
            if (dep[head[u]] < dep[head[v]])
                swap(u, v);
            ans = max(ans, query(pos[head[u]], pos[u]));
            u = p[head[u]];
        }
        if (dep[u] > dep[v])
            swap(u, v);
        cout << max(ans, query(pos[u], pos[v])) << '\n';
    }
}
}

```

```

import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]));g=[[]for _ in range(n)];j=2+n
    for _ in range(n-1):u,v=int(d[j])-1,int(d[j+1])-1;j+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;dep=[0]*n;order=[0]
    for u in order:
        for v in g[u]:
            if v!=par[u]:par[v]=u;dep[v]=dep[u]+1;order.append(v)
    size=[1]*n;heavy=[-1]*n
    for u in reversed(order[1:]):
        size[par[u]]+=size[u]
        if heavy[par[u]]<0 or size[u]>size[heavy[par[u]]]:heavy[par[u]]=u
    head=[0]*n;pos=[0]*n;timer=0;todo=[(0,0)]
    while todo:
        u,h=todo.pop()
        while u!=-1:
            head[u]=h;pos[u]=timer;timer+=1
            for v in g[u]:
                if v!=par[u] and v!=heavy[u]:todo.append((v,v))
            u=heavy[u]
    z=1
    while z<n:z*=2
    tree=[-10**30]*(2*z)
    for u in range(n):tree[z+pos[u]]=a[u]
    for i in range(z-1,0,-1):tree[i]=max(tree[2*i],tree[2*i+1])
    def rq(l,r):
        ans=-10**30;l+=z;r+=z+1
        while l<r:
            if l&1:ans=max(ans,tree[l]);l+=1
            if r&1:r-=1;ans=max(ans,tree[r])
            l//=2;r//=2
        return ans
    out=[]
    for _ in range(q):
        c=d[j];u=int(d[j+1])-1;v=int(d[j+2]);j+=3
        if c==b'U':
            x=z+pos[u];tree[x]=v;x//=2
            while x:tree[x]=max(tree[2*x],tree[2*x+1]);x//=2
        else:
            v-=1;ans=-10**30
            while head[u]!=head[v]:
                if dep[head[u]]<dep[head[v]]:u,v=v,u
                ans=max(ans,rq(pos[head[u]],pos[u]));u=par[head[u]]
                if dep[u]>dep[v]:u,v=v,u
            out.append(str(max(ans,rq(pos[u],pos[v]))))
    print('\n'.join(out))
if __name__=="__main__":main()

```

## 10 G. Subtree Value Count

Flatten the tree in preorder, making a subtree a contiguous interval. Build a merge-sort tree: every node stores the sorted values in its positional segment. Decompose a query interval into  $O(\log n)$  nodes and use two binary searches in each to count values in  $[lo, hi]$ . The selected nodes partition the subtree interval, so their counts neither omit nor duplicate a vertex. Time is  $O(\log^2 n)$  per query and memory  $O(n \log n)$ .

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> tin(n), tout(n), par(n, -1), order, st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        order.push_back(u);
        for (int v : g[u])
            if (v != par[u])
                par[v] = u, st.push_back(v);
    }
    for (int i = 0; i < n; i++)
        tin[order[i]] = i;
    vector<int> sz(n, 1);
    for (int i = n - 1; i; i--)
        sz[par[order[i]]] += sz[order[i]];
    for (int u = 0; u < n; u++)
        tout[u] = tin[u] + sz[u];
    int z = 1;
    while (z < n)
        z *= 2;
```

```

vector<vector<int>> t(2 * z);
for (int u = 0; u < n; u++)
    t[z + tin[u]] = {a[u]};
for (int i = z - 1; i; i--) {
    auto &x = t[2 * i];
    auto &y = t[2 * i + 1];
    t[i].resize(x.size() + y.size());
    merge(x.begin(), x.end(), y.begin(), y.end(), t[i].begin());
}
while (q--) {
    int u, lo, hi;
    cin >> u >> lo >> hi;
    --u;
    int l = z + tin[u], r = z + tout[u], ans = 0;
    while (l < r) {
        if (l & 1) {
            auto &v = t[l++];
            ans += upper_bound(v.begin(), v.end(), hi) -
                    lower_bound(v.begin(), v.end(), lo);
        }
        if (r & 1) {
            auto &v = t[--r];
            ans += upper_bound(v.begin(), v.end(), hi) -
                    lower_bound(v.begin(), v.end(), lo);
        }
        l /= 2;
        r /= 2;
    }
    cout << ans << '\n';
}
}

```

```

import bisect,sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,q=d[:2];a=d[2:2+n];g=[[[]for _ in range(n)];j=2+n
    for _ in range(n-1):u,v=d[j]-1,d[j+1]-1;j+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;order=[];stack=[0]
    while stack:
        u=stack.pop();order.append(u)
        for v in g[u]:
            if v!=par[u]:par[v]=u;stack.append(v)
    tin=[0]*n
    for i,u in enumerate(order):tin[u]=i
    size=[1]*n
    for u in reversed(order[1:]):size[par[u]]+=size[u]
    z=1
    while z<n:z*=2
    tree=[[[]for _ in range(2*z)]
    for u in range(n):tree[z+tin[u]]=a[u]]

```

```

for i in range(z-1,0,-1):tree[i]=sorted(tree[2*i]+tree[2*i+1])
out=[]
for _ in range(q):
    u,lo,hi=d[j]-1,d[j+1],d[j+2];j+=3;l=z+tin[u];r=l+size[u];ans=0
    while l<r:
        if l&1:v=tree[l];ans+=bisect.bisect_right(v,hi)-bisect.bisect_left(v,lo);l+=1
        if r&1:r-=1;v=tree[r];ans+=bisect.bisect_right(v,hi)-bisect.bisect_left(v,lo)
        l//=2;r//=2
    out.append(str(ans))
print('\n'.join(out))
if __name__=="__main__":main()

```

## 11 H. Connectivity Countdown

DSU cannot delete edges, but reversing time turns every deletion into an addition. First mark all edges ever deleted and build a DSU from the edges that survive to the end. Scan operations backward: answer a query in the current DSU, and undo a forward deletion by joining that edge.

Before each reversed operation, the DSU contains exactly the edges present immediately after that operation in forward time. This is true at the end by construction; reversing a deletion adds exactly its one missing edge, while a query changes nothing. Induction proves every recorded answer correct. The total time is  $O((n+m+q) \alpha(n))$  and memory is  $O(n+m+q)$ .

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;
    DSU(int n) : parent(n), size(n, 1) { iota(parent.begin(), parent.end(), 0); }
    int find(int u) { return parent[u] == u ? u : parent[u] = find(parent[u]); }
    void join(int u, int v) {
        u = find(u); v = find(v);
        if (u == v) return;
        if (size[u] < size[v]) swap(u, v);
        parent[v] = u; size[u] += size[v];
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, q;
    if (!(cin >> n >> m >> q)) return 0;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) { cin >> u >> v; --u; --v; }
    struct Operation { char type; int x, y; };
    vector<Operation> operations(q);
    vector<char> deleted(m);
    for (auto &[type, x, y] : operations) {
        cin >> type >> x; --x;
        if (type == 'D') { y = -1; deleted[x] = true; }
        else { cin >> y; --y; }
    }
    DSU dsu(n);
    for (int e = 0; e < m; ++e)
        if (!deleted[e]) dsu.join(edges[e].first, edges[e].second);
    vector<string> answers;
    for (int i = q - 1; i >= 0; --i) {
```

```

    auto [type, x, y] = operations[i];
    if (type == 'D') dsu.join(edges[x].first, edges[x].second);
    else answers.push_back(dsu.find(x) == dsu.find(y) ? "YES" : "NO");
}
reverse(answers.begin(), answers.end());
for (const string &answer : answers) cout << answer << '\n';
}

```

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n, m, q = map(int, data[:3])
    index = 3
    edges = []
    for _ in range(m):
        u, v = map(int, data[index:index+2]); index += 2
        edges.append((u-1, v-1))
    operations = []
    deleted = [False] * m
    for _ in range(q):
        operation = data[index]; index += 1
        if operation == b'D':
            edge = int(data[index]) - 1; index += 1
            operations.append((operation, edge, -1)); deleted[edge] = True
        else:
            u, v = map(int, data[index:index+2]); index += 2
            operations.append((operation, u-1, v-1))
    parent = list(range(n)); size = [1] * n
    def find(u):
        while parent[u] != u:
            parent[u] = parent[parent[u]]; u = parent[u]
        return u
    def join(u, v):
        u, v = find(u), find(v)
        if u == v: return
        if size[u] < size[v]: u, v = v, u
        parent[v] = u; size[u] += size[v]
    for edge, (u, v) in enumerate(edges):
        if not deleted[edge]: join(u, v)
    answers = []
    for operation, x, y in reversed(operations):
        if operation == b'D': join(*edges[x])
        else: answers.append('YES' if find(x) == find(y) else 'NO')
    print('\n'.join(reversed(answers)))

if __name__ == "__main__":
    main()

```

## 12 I. Persistent Version Sums

A point assignment changes only one root-to-leaf path of a segment tree. Create new nodes on that path and share every untouched subtree with the old root. Each root therefore describes one immutable version. Induction on a node's interval shows its sum uses the updated child exactly when the position lies there and otherwise uses the old child, so the new root represents the requested copied-and-edited array. Build time is  $O(n)$ ; every update and query is  $O(\log n)$ ; memory is  $O(n+q \log n)$ .

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct N {
    int l = 0, r = 0;
    ll s = 0;
};
vector<N> t(1);
int build(int l, int r, vector<ll> &a) {
    int x = t.size();
    t.push_back({});
    if (r - l == 1) {
        t[x].s = a[l];
        return x;
    }
    int m = (l + r) / 2;
    t[x].l = build(l, m, a);
    t[x].r = build(m, r, a);
    t[x].s = t[t[x].l].s + t[t[x].r].s;
    return x;
}
int update(int old, int l, int r, int p, ll v) {
    int x = t.size();
    t.push_back(t[old]);
    if (r - l == 1) {
        t[x].s = v;
        return x;
    }
    int m = (l + r) / 2;
    if (p < m)
        t[x].l = update(t[old].l, l, m, p, v);
    else
        t[x].r = update(t[old].r, m, r, p, v);
    t[x].s = t[t[x].l].s + t[t[x].r].s;
    return x;
}
ll query(int x, int l, int r, int ql, int qr) {
```

```

    if (ql >= r || qr <= l)
        return 0;
    if (ql <= l && r <= qr)
        return t[x].s;
    int m = (l + r) / 2;
    return query(t[x].l, l, m, ql, qr) + query(t[x].r, m, r, ql, qr);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    vector<int> root = {build(0, n, a)};
    while (q--) {
        char c;
        int v, l;
        cin >> c >> v >> l;
        if (c == 'U') {
            ll x;
            cin >> x;
            root.push_back(update(root[v], 0, n, l - 1, x));
        } else {
            int r;
            cin >> r;
            cout << query(root[v], 0, n, l - 1, r) << '\n';
        }
    }
}

```

```

from array import array
import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]))
    left=array('i',[0]);right=array('i',[0]);total=array('q',[0])
    def node(l=0,r=0,s=0):left.append(l);right.append(r);total.append(s);return len(total)-1
    def build(l,r):
        if r-l==1:return node(s=a[l])
        m=(l+r)//2;x=node(build(l,m),build(m,r));total[x]=total[left[x]]+total[right[x]];ret
    def update(old,l,r,p,v):
        x=node(left[old],right[old],total[old])
        if r-l==1:total[x]=v;return x
        m=(l+r)//2
        if p<m:left[x]=update(left[old],l,m,p,v)
        else:right[x]=update(right[old],m,r,p,v)
        total[x]=total[left[x]]+total[right[x]];return x

```

```

def query(x,l,r,ql,qr):
    if ql>=r or qr<=l:return 0
    if ql<=l and r<=qr:return total[x]
    m=(l+r)//2;return query(left[x],l,m,ql,qr)+query(right[x],m,r,ql,qr)
roots=[build(0,n)];j=2+n;out=[]
for _ in range(q):
    c=d[j];v=int(d[j+1]);l=int(d[j+2]);j+=3
    if c==b'U':x=int(d[j]);j+=1;roots.append(update(roots[v],0,n,l-1,x))
    else:r=int(d[j]);j+=1;out.append(str(query(roots[v],0,n,l-1,r)))
print('\n'.join(out))
if __name__=="__main__":main()

```

## 13 J. Subtree K-th Smallest

Flatten the tree and build persistent frequency roots for successive prefixes of the Euler array. The multiset of subtree  $[l, r]$  is the frequency difference between roots  $r$  and  $l$ . At each value-tree node, subtract the left-child counts: if at least  $k$  elements lie left, descend left; otherwise subtract that count and descend right. This preserves the rank of the target until its compressed value leaf is reached. Preprocessing and memory are  $O(n \log n)$ ; each query is  $O(\log n)$ .

```
#include <bits/stdc++.h>
using namespace std;
struct N {
    int l = 0, r = 0, s = 0;
};
vector<N> t(1);
int add(int old, int l, int r, int p) {
    int x = t.size();
    t.push_back(t[old]);
    t[x].s++;
    if (r - l > 1) {
        int m = (l + r) / 2;
        if (p < m)
            t[x].l = add(t[old].l, l, m, p);
        else
            t[x].r = add(t[old].r, m, r, p);
    }
    return x;
}
int kth(int a, int b, int l, int r, int k) {
    if (r - l == 1)
        return l;
    int c = t[t[b].l].s - t[t[a].l].s, m = (l + r) / 2;
    return k <= c ? kth(t[a].l, t[b].l, l, m, k)
        : kth(t[a].r, t[b].r, m, r, k - c);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<int> a(n), vals;
    for (int &x : a)
        cin >> x;
    vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
```

```

vector<vector<int>> g(n);
for (int i = 1, u, v; i < n; i++) {
    cin >> u >> v;
    --u;
    --v;
    g[u].push_back(v);
    g[v].push_back(u);
}
vector<int> p(n, -1), order, st = {0};
while (!st.empty()) {
    int u = st.back();
    st.pop_back();
    order.push_back(u);
    for (int v : g[u])
        if (v != p[u])
            p[v] = u, st.push_back(v);
}
vector<int> tin(n), sz(n, 1);
for (int i = 0; i < n; i++)
    tin[order[i]] = i;
for (int i = n - 1; i; i--)
    sz[p[order[i]]] += sz[order[i]];
vector<int> root(n + 1);
for (int i = 0; i < n; i++) {
    int u = order[i],
        x = lower_bound(vals.begin(), vals.end(), a[u]) - vals.begin();
    root[i + 1] = add(root[i], 0, vals.size(), x);
}
while (q--) {
    int u, k;
    cin >> u >> k;
    --u;
    cout << vals[kth(root[tin[u]], root[tin[u] + sz[u]], 0, vals.size(), k)]
        << '\n';
}
}

```

```

from array import array
import bisect, sys
def main():
    d=list(map(int,sys.stdin.buffer.read().split()))
    if not d:return
    n,q=d[:2];a=d[2:2+n];vals=sorted(set(a));g=[[[]for _ in range(n)];j=2+n
    for _ in range(n-1):u,v=d[j]-1,d[j+1]-1;j+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;order=[];stack=[0]
    while stack:
        u=stack.pop();order.append(u)
        for v in g[u]:
            if v!=par[u]:par[v]=u;stack.append(v)
    tin=[0]*n;size=[1]*n
    for i,u in enumerate(order):tin[u]=i

```

```

for u in reversed(order[1:]):size[par[u]]+=size[u]
left=array('i',[0]);right=array('i',[0]);cnt=array('i',[0])
def add(old,l,r,p):
    left.append(left[old]);right.append(right[old]);cnt.append(cnt[old]+1);x=len(cnt)-1
    if r-l>1:
        m=(l+r)//2
        if p<m:left[x]=add(left[old],l,m,p)
        else:right[x]=add(right[old],m,r,p)
    return x
roots=[0]
for u in order:roots.append(add(roots[-1],0,len(vals),bisect.bisect_left(vals,a[u])))
def kth(x,y,l,r,k):
    while r-l>1:
        m=(l+r)//2;c=cnt[left[y]]-cnt[left[x]]
        if k<=c:x,y,r=left[x],left[y],m
        else:k-=c;x,y,l=right[x],right[y],m
    return l
out=[]
for _ in range(q):
    u,k=d[j]-1,d[j+1];j+=2
    rank=kth(roots[tin[u]],roots[tin[u]+size[u]],0,len(vals),k)
    out.append(str(vals[rank]))
print('\n'.join(out))
if __name__=="__main__":main()

```

## 14 K. Sparse Rectangle Sums

Collect all update points before processing. Compress their x-coordinates. For each outer Fenwick index, store only the y-coordinates of updates that can reach that index, with an inner Fenwick tree over those coordinates. A point update follows both Fenwick directions; `prefix(x,y)` sums precisely points with both coordinates bounded. Inclusion–exclusion of four prefixes gives an inclusive rectangle. Each operation costs  $O(\log^2 q)$  and the nested trees use  $O(q \log q)$  memory. Query boundaries need not occur among update coordinates: `upper_bound` handles them.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Op {
    char c;
    ll a, b, c1, d;
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q))
        return 0;
    vector<Op> ops;
    vector<ll> xs;
    for (int i = 0; i < q; i++) {
        char c;
        cin >> c;
        if (c == 'U') {
            ll x, y, d;
            cin >> x >> y >> d;
            ops.push_back({c, x, y, d, 0});
            xs.push_back(x);
        } else {
            ll x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            ops.push_back({c, x1, y1, x2, y2});
        }
    }
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
    int n = xs.size();
    vector<vector<ll>> ys(n + 1), bit(n + 1);
    for (auto &o : ops)
        if (o.c == 'U') {
            int x = lower_bound(xs.begin(), xs.end(), o.a) - xs.begin() + 1;
```

```

        for (int i = x; i <= n; i += i & -i)
            ys[i].push_back(o.b);
    }
    for (int i = 1; i <= n; i++) {
        sort(ys[i].begin(), ys[i].end());
        ys[i].erase(unique(ys[i].begin(), ys[i].end()), ys[i].end());
        bit[i].assign(ys[i].size() + 1, 0);
    }
    auto upd = [&](ll x, ll y, ll d) {
        for (int i = lower_bound(xs.begin(), xs.end(), x) - xs.begin() + 1; i <= n;
            i += i & -i)
            for (int j =
                lower_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin() + 1;
                j < (int)bit[i].size(); j += j & -j)
                bit[i][j] += d;
    };
    auto pref = [&](ll x, ll y) {
        ll ans = 0;
        for (int i = upper_bound(xs.begin(), xs.end(), x) - xs.begin(); i;
            i -= i & -i)
            for (int j = upper_bound(ys[i].begin(), ys[i].end(), y) - ys[i].begin();
                j; j -= j & -j)
                ans += bit[i][j];
        return ans;
    };
    for (auto &o : ops)
        if (o.c == 'U')
            upd(o.a, o.b, o.c1);
        else
            cout << pref(o.c1, o.d) - pref(o.a - 1, o.d) - pref(o.c1, o.b - 1) +
                pref(o.a - 1, o.b - 1)
                << '\n';
}

```

```

import bisect,sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    q=int(d[0]);j=1;ops=[];xs=[]
    for _ in range(q):
        c=d[j];j+=1
        if c==b'U':x,y,v=map(int,d[j:j+3]);j+=3;ops.append((c,x,y,v));xs.append(x)
        else:x1,y1,x2,y2=map(int,d[j:j+4]);j+=4;ops.append((c,x1,y1,x2,y2))
    xs=sorted(set(xs));n=len(xs);ys=[[[]for _ in range(n+1)]
    for o in ops:
        if o[0]==b'U':
            i=bisect.bisect_left(xs,o[1])+1
            while i<=n:ys[i].append(o[2]);i+=i&-i
    ys=[sorted(set(v)) for v in ys];bits=[[0]*(len(v)+1) for v in ys]
    def update(x,y,v):
        i=bisect.bisect_left(xs,x)+1

```

```

        while i<=n:
            k=bisect.bisect_left(ys[i],y)+1
            while k<len(bits[i]):bits[i][k]+=v;k+=k&-k
            i+=i&-i
def pref(x,y):
    ans=0;i=bisect.bisect_right(xs,x)
    while i:
        k=bisect.bisect_right(ys[i],y)
        while k:ans+=bits[i][k];k-=k&-k
        i-=i&-i
    return ans
out=[]
for o in ops:
    if o[0]==b'U':update(o[1],o[2],o[3])
    else:
        _,x1,y1,x2,y2=o;out.append(str(pref(x2,y2)-pref(x1-1,y2)-pref(x2,y1-1)+pref(x1-1,y1)))
print('\n'.join(out))
if __name__=="__main__":main()

```

## 15 L. Path Add, Path Maximum

Use heavy-light decomposition as in F, but place a lazy range-add/range-max segment tree on the base array. Split each requested path into chain segments; apply the range addition to every piece or take the maximum of their range maxima. The HLD pieces partition exactly the path, and the lazy tree invariant says a node maximum already includes every addition covering that node. Together these facts prove both operations correct.

There are  $O(\log n)$  pieces, each costing  $O(\log n)$ , so every operation is  $O(\log^2 n)$  and memory is  $O(n)$ . Negative values require a genuine negative infinity neutral element, not zero.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Seg {
    int n;
    vector<ll> mx, lz;
    Seg(vector<ll> &a) {
        n = a.size();
        mx.resize(4 * n);
        lz.resize(4 * n);
        function<void(int, int, int)> b = [&](int x, int l, int r) {
            if (r - l == 1) {
                mx[x] = a[l];
                return;
            }
            int m = (l + r) / 2;
            b(2 * x, l, m);
            b(2 * x + 1, m, r);
            mx[x] = max(mx[2 * x], mx[2 * x + 1]);
        };
        b(1, 0, n);
    }
    void add(int x, int l, int r, int ql, int qr, ll v) {
        if (ql >= r || qr <= l)
            return;
        if (ql <= l && r <= qr) {
            mx[x] += v;
            lz[x] += v;
            return;
        }
        int m = (l + r) / 2;
        add(2 * x, l, m, ql, qr, v);
        add(2 * x + 1, m, r, ql, qr, v);
        mx[x] = lz[x] + max(mx[2 * x], mx[2 * x + 1]);
    }
};
```

```

ll get(int x, int l, int r, int ql, int qr, ll carry = 0) {
    if (ql >= r || qr <= l)
        return LLONG_MIN;
    if (ql <= l && r <= qr)
        return mx[x] + carry;
    int m = (l + r) / 2;
    return max(get(2 * x, l, m, ql, qr, carry + lz[x]),
               get(2 * x + 1, m, r, ql, qr, carry + lz[x]));
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> p(n, -1), dep(n), sz(n, 1), heavy(n, -1), head(n), pos(n),
               ord = {0};
    for (int i = 0; i < (int)ord.size(); i++) {
        int u = ord[i];
        for (int v : g[u])
            if (v != p[u])
                p[v] = u, dep[v] = dep[u] + 1, ord.push_back(v);
    }
    for (int z = n - 1; z; z--) {
        int u = ord[z];
        sz[p[u]] += sz[u];
        if (heavy[p[u]] < 0 || sz[u] > sz[heavy[p[u]]])
            heavy[p[u]] = u;
    }
    int timer = 0;
    vector<pair<int, int>> todo = {{0, 0}};
    while (!todo.empty()) {
        auto [u, h] = todo.back();
        todo.pop_back();
        for (int x = u; x != -1; x = heavy[x]) {
            head[x] = h;
            pos[x] = timer++;
            for (int v : g[x])
                if (v != p[x] && v != heavy[x])

```

```

        todo.push_back({v, v});
    }
}
vector<ll> flat(n);
for (int u = 0; u < n; u++)
    flat[pos[u]] = a[u];
Seg s(flat);
auto each = [&](int u, int v, auto f) {
    while (head[u] != head[v]) {
        if (dep[head[u]] < dep[head[v]])
            swap(u, v);
        f(pos[head[u]], pos[u] + 1);
        u = p[head[u]];
    }
    if (dep[u] > dep[v])
        swap(u, v);
    f(pos[u], pos[v] + 1);
};
while (q--) {
    char c;
    int u, v;
    cin >> c >> u >> v;
    --u;
    --v;
    if (c == 'A') {
        ll x;
        cin >> x;
        each(u, v, [&](int l, int r) { s.add(1, 0, n, l, r, x); });
    } else {
        ll ans = LLONG_MIN;
        each(u, v, [&](int l, int r) { ans = max(ans, s.get(1, 0, n, l, r)); });
        cout << ans << '\n';
    }
}
}
}

```

```

import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]));g=[[]for _ in range(n)];j=2+n
    for _ in range(n-1):u,v=int(d[j])-1,int(d[j+1])-1;j+=2;g[u].append(v);g[v].append(u)
    par=[-1]*n;dep=[0]*n;order=[0]
    for u in order:
        for v in g[u]:
            if v!=par[u]:par[v]=u;dep[v]=dep[u]+1;order.append(v)
    size=[1]*n;heavy=[-1]*n
    for u in reversed(order[1:]):
        size[par[u]]+=size[u]
        if heavy[par[u]]<0 or size[u]>size[heavy[par[u]]]:heavy[par[u]]=u
    head=[0]*n;pos=[0]*n;timer=0;todo=[(0,0)]

```

```

while todo:
    u,h=todo.pop()
    while u!=-1:
        head[u]=h;pos[u]=timer;timer+=1
        for v in g[u]:
            if v!=par[u] and v!=heavy[u]:todo.append((v,v))
        u=heavy[u]
flat=[0]*n
for u in range(n):flat[pos[u]]=a[u]
mx=[0]*(4*n);lazy=[0]*(4*n)
def build(x,l,r):
    if r-l==1:mx[x]=flat[l];return
    m=(l+r)//2;build(2*x,l,m);build(2*x+1,m,r);mx[x]=max(mx[2*x],mx[2*x+1])
def add(x,l,r,ql,qv,v):
    if ql>=r or qv<=l:return
    if ql<=l and r<=qv:mx[x]+=v;lazy[x]+=v;return
    m=(l+r)//2;add(2*x,l,m,ql,qv,v);add(2*x+1,m,r,ql,qv,v);mx[x]=lazy[x]+max(mx[2*x],mx[2*x+1])
def get(x,l,r,ql,qv,carry=0):
    if ql>=r or qv<=l:return -10**30
    if ql<=l and r<=qv:return mx[x]+carry
    m=(l+r)//2;carry+=lazy[x];return max(get(2*x,l,m,ql,qv,carry),get(2*x+1,m,r,ql,qv,carry))
build(1,0,n);out=[]
def parts(u,v):
    while head[u]!=head[v]:
        if dep[head[u]]<dep[head[v]]:u,v=v,u
        yield pos[head[u]],pos[u]+1;u=par[head[u]]
    if dep[u]>dep[v]:u,v=v,u
    yield pos[u],pos[v]+1
for _ in range(q):
    c=d[j];u=int(d[j+1])-1;v=int(d[j+2])-1;j+=3
    if c==b'A':
        value=int(d[j]);j+=1
        for l,r in parts(u,v):add(1,0,n,l,r,value)
    else:out.append(str(max(get(1,0,n,l,r) for l,r in parts(u,v))))
print('\n'.join(out))
if __name__=="__main__":main()

```

## 16 Testing discussion

Run `CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py`. For small random cases, compare every new structure with its direct model: explicit subtree/path updates, BFS connectivity, copied version arrays, sorted subtree values, and a point dictionary for rectangle sums. These oracles expose incorrect Euler orders, rollback boundaries, persistence sharing errors, and two-dimensional compression mistakes.

## 17 What to carry forward

Fenwick trees are prefix-summary engines. Pair them when a range update needs an extra weighted correction, binary-lift them when a prefix count becomes an order statistic, and combine one with a sweep when one coordinate can be made monotone.

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.